

Supplementary Material for the paper “A Taxonomy of Test Case Breakages and Repairs in the Web Application Domain”

I. INFRASTRUCTURE

An infrastructure is developed to collect, manage, and analyze test case breakage data. The data gathered in the process has a complex structure. It was inefficient to store the data in digital spreadsheets or to do the analysis afterwards. So, we needed an infrastructure to manage the data.

The infrastructure has a full-stack web application architecture. It has a relational database to store the data. A backend is used to interact with the database, and a frontend is used for users to interact with. This web application makes it easier to collect, manage, and query the data. In Figure 1, we see the relationship between the database, backend, and frontend.

II. DATABASE

The database stores the data collected during the process. It helps manage and structure the data. It also makes it possible to easily inspect the result by simply querying it.

A MySQL relational database was selected for its reliability and strong integration with the Spring ecosystem. The database is managed through Spring Data JPA. JPA allows the backend to interact with the database using object relational mapping (ORM). ORM maps the Java objects to the database tables and vice versa. JPA simplifies the creation, updating, deleting, and retrieval of data. Manual SQL queries are not needed to perform these operations, but custom queries are also allowed when necessary.

Spring initializes and updates the schema automatically after running the application. It uses entity and other annotations to create the tables and relationships. This setup ensures that any researcher or developer can reproduce the exact schema. Simply starting the application will produce the same first state of the system.

The schema has 18 tables. Figure 2 shows the relationship between these tables.

a) $t_application$: The application table stores information about the applications. It has `id`, `app_name`, and `url` columns. For this and the other tables, the `id` is the primary key and identifies the single record. The `appName` column stores the application name, and the `url` column has the URL of the GitHub page of the applications. Every application can have one or more app releases (versions) and test suites. Every application can have programming languages that are used to write the application.

b) $t_app_release$: The application release table stores information about the application versions. It has `id`, `release_name`, and `application_id` columns. The `release_name` column stores the version number, e.g., 2.3.4, 3.13.1. The `application_id` column is the foreign key. It refers to the primary key in the application table. It states to which application the release record belongs. An app release can have one more breakage.

c) t_test_suite : The test suite table stores information about the test suites. It has `id`, `name`, `url`, and `application_id` columns. `name` stores the name of the test suite. `url` stores the GitHub URL that contains the repair commits of the tests in the test suite. The `application_id` column is the foreign key. It refers to the primary key in the application table. It states to which application the test suite record belongs. A test suite can have one or more test cases.

d) t_test_case : The test case table stores information about the test cases. It has `id`, `name`, `description`, and `test_suite_id` columns. `name` stores the test case name. `description` column stores details about the test case. The `test_suite_id` column is the foreign key. It refers to the primary key in the test suite table. It states to which test suite the test case record belongs. A test case can have one or more test case versions.

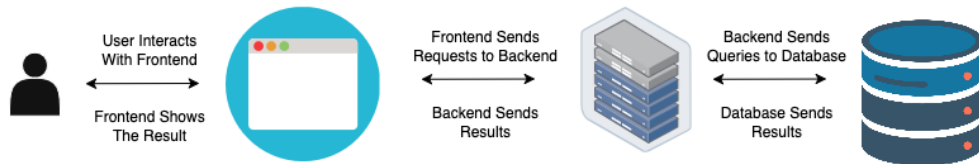


Fig. 1: Database, backend, and frontend interact with each other.

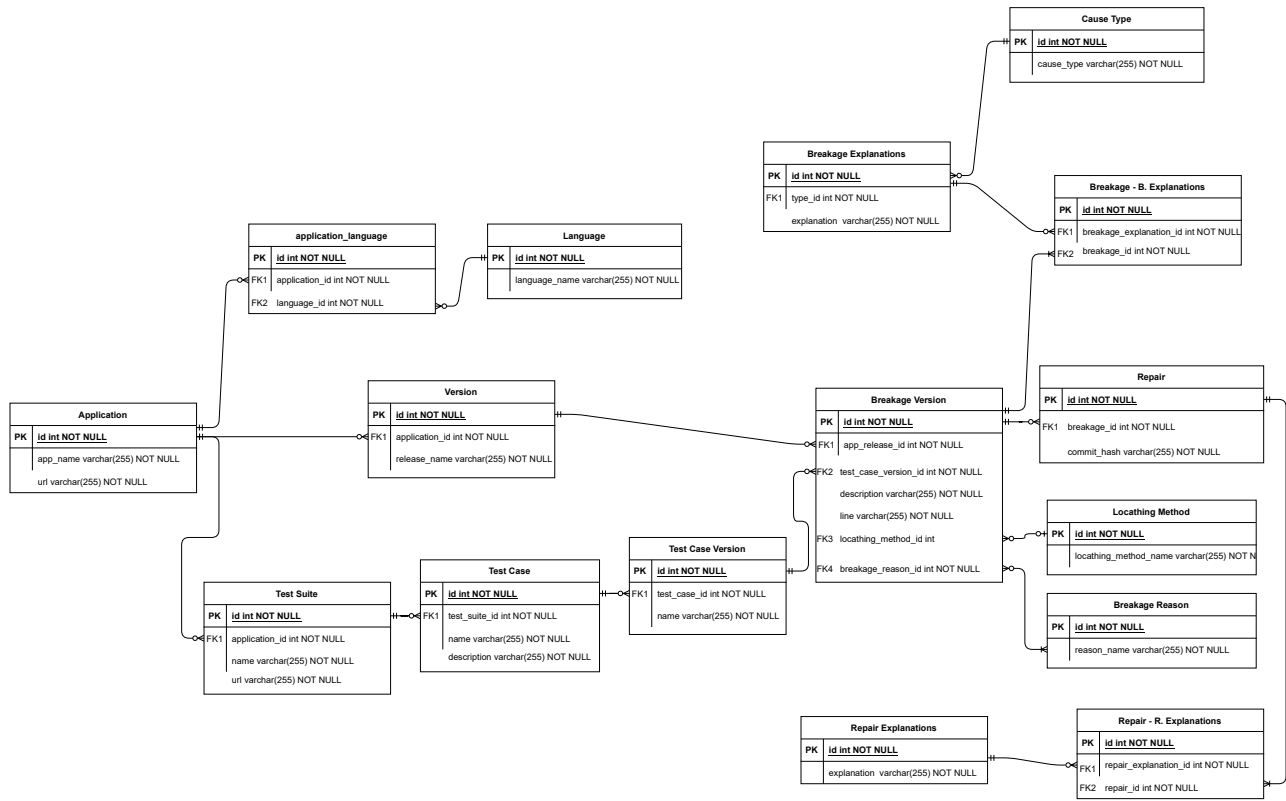


Fig. 2: Entity relationship diagram

e) *t_test_case_version*: : The test case table stores information about the test case versions. The reason for this table is that the test case and the test release of the breakage are not the same, and we need to determine which version of the test case ran when the breakage was produced. It has *id*, *name*, and *test_case_id* columns. *name* stores the test case version name, e.g., AddContentTest 2.3.4. The *test_case_id* column is the foreign key. It refers to the primary key in the test case table. It states which test case the test case version record belongs to. A test case can have one or more breakages.

f) *t_breakage*: : The breakage table stores information about the breakages. It has *id*, *description*, *line*, *appReleaseId*, *testCaseVersionId*, *breakageReasonId*, *locatingMethodId* columns. The *description* column stores error messages e.g. `org.openqa.selenium.NoSuchElementException: no such element: Unable to locate element: {"method":"css selector","selector":- "#jsSaveDraft"}`. *line* column stores the line in which the error occurred, e.g., at `tests.AddAsDraftTest.-addAsDraft(AddAsDraftTest.java:18)`. *appReleaseId*, *testCaseVersionId*, *breakageReasonId*, *locatingMethodId* store foreign keys from the corresponding tables. A breakage can have repairs and breakage explanations.

g) *t_repair*: : The test repair table stores information about the repairs. It has *id*, *commit_hash*, and *breakage_id* columns. *commit_hash* column stores the commit hash that has the repair of the breakage. The commit should be inside the GitHub test suite repository to show the commit changes in the web app later. *breakage_id* column is the foreign key. It refers to the primary key in the breakage table. It states to which breakage belongs the repair record. A repair can have repair explanations.

h) *t_breakage_explanation*: : The test breakage explanation table stores information about the breakage explanations. It has *id*, *explanation* and *typeId* columns. The *explanation* column has the breakage explanations, e.g., Id Attribute Not Found, Unexpected Assertion Value. The *typeId* column is the foreign key. It refers to the primary key in the cause type table. It states whether the explanation is a proximal or distal cause. A breakage explanation can have one or more breakages.

i) *t_repair_explanation*: : The test repair explanation table stores information about the repair explanations. It has *id* and *explanation* columns. The *explanation* column has the repair explanations, e.g., Locator Fixed. A repair explanation can have one or more repairs.

j) *t_breakage_reason*: : This is a generalization of the *description* column in the breakage table for easy querying. It has *id*, and *reason_name* columns. The *reason_name* column has the breakage reason, e.g., `org.openqa.selenium.NoSuchElementException: no such element: Unable to locate element`. A breakage reason can have one or more breakages.

k) *t_locating_method*: : The locating method table stores information about the locating methods. It has *id*, and *locating_method_name* columns. The *locating_method_name* column has the locating methods, e.g., XPath, CSS selector. A locating method can have one or more breakages.

l) *t_language*: : The language table stores information about the languages. It has *id*, and *language_name* columns. The *language_name* column has the languages, e.g., Java, PHP. A language can have one or more applications.

m) *breakage_explanation*: : This table stores the relationship between breakages and breakage explanations. It has *breakageId* and *breakageExplanationId* columns. For example, a record with 1, 2 would mean that the breakage with id 1 can be explained by the breakage explanation with 2.

n) *repair_explanation*: : This table stores the relationship between repairs and repair explanations. It has *repairId* and *repairExplanationId* columns. For example, a record with 1, 2 would mean that the repair with id 1 can be explained by the repair explanation with 2.

o) *t_application_language*: : This table stores the relationship between applications and languages. It has *applicationId* and *languageId* columns. For example, a record with 1, 2 would mean that the application with id 1 is written by the programming language with id 2.

p) *t_cause_type*: : The cause type table stores information about the types of causes. It has *id*, and *type* columns. The *type* could be either distal or proximal. A cause type can have one or more breakage explanations.

q) : The database in this project stores the data in a structured manner. It helps in managing them and simplifies their analysis. It also makes it possible to easily control the results and the taxonomy.

III. BACKEND

The backend of the web application is implemented using the Spring Framework. The backend interacts with the database and has the business logic of the application. It processes data, enforces the business logic, and exposes a clean interface. The backend has three main responsibilities.

1) Role and Responsibilities:

a) *API Layer*: The Backend provides RESTful endpoints to interact with the application. These endpoints can be used by frontend or external services. They are used to save, retrieve, and manipulate data, such as test breakage information. Endpoints are implemented using Spring Web MVC and follow REST conventions for clarity and scalability.

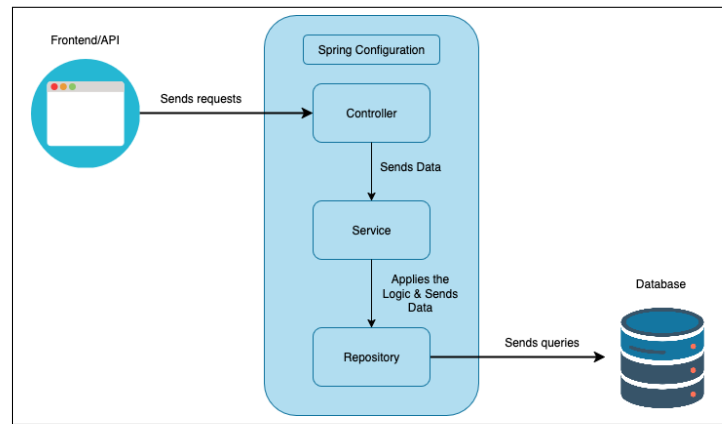


Fig. 3: Controller accepts the request and sends the data to service, which then send to repository layer and then to the database.

b) Business Logic: The backend not only handles data, but also forces the business logic. For example, when a breakage information is requested from the frontend, the service layer checks the repair commit hash and retrieves the commit from the GitHub repository.

c) Persistence Layer: The backend communicates with the MySQL database using Spring Data JPA. It uses Java entity classes to create the database tables and communicates with them using repository interfaces. This enables a clean separation of concerns. It allows declarative data access, but also supports advanced custom queries.

2) *Architecture and Design:* The backend has a layered architecture that separate concerns. Figure 3 shows the flow between the layers.

a) Controller Layer: Handles HTTP requests and responses. Controllers receives the request and delegates processing to the service layer. For example, in listing 1 PostMapping method takes the body of the POST request that is sent to /api/applications. Then sends it to the service layer to create a new application.

```

@RestController
@RequestMapping("/api/applications")
public class ApplicationController {

    private final ApplicationService applicationService;

    @PostMapping
    public ResponseEntity<ApplicationDTO> save(@RequestBody ApplicationDTO application) {
        return ResponseEntity.status(HttpStatus.CREATED).body(applicationService.save(
            application));
    }
}

```

Listing 1: Controller takes the body with new application information and sends it to the service layer

b) Service Layer: Has the business logic. It coordinates between controllers and the repository. It handles errors, does the data transformation (DTOs). For example, in listing 2 save method takes the ApplicationDTO sent from the controller, creates an application using it, and sends the newly created application to the repository layer.

```

public ApplicationDTO save(ApplicationDTO applicationDTO) {
    Application application = new Application();
    application.setAppName(applicationDTO.getAppName());
    application.setUrl(applicationDTO.getUrl());
    application.setLanguages(applicationDTO.getLanguages().stream()
        .map(dto -> languageRepository.getReferenceById(dto.getId()))
        .collect(Collectors.toList()));
    applicationRepository.save(application);

    applicationDTO.setId(application.getId());
    return applicationDTO;
}

```

Listing 2: ApplicationService, creates a Application object and sends it to the repository layer

c) *Repository Layer*: Interacts with the database using the JPA repository. It supports both simple CRUD operations and complex queries. For example, in listing 3 `ApplicationRepository` extends `JpaRepository`. Thus, there is no need for an explicit `save` method implementation. Repository takes the `Application` object sent from above and saves it to the database.

```
@Repository
public interface ApplicationRepository extends JpaRepository<Application, Integer> {
    public Application getReferenceById(int id);
}
```

Listing 3: Repository layer extends `JpaRepository`

d) *Entity Layer*: Using Java objects and Java Persistence API (JPA) annotations defines the model. Entities form the structure of the database schema and tables. For example, in listing 4 `Entity` annotation used to form the entity `Application` in the database. Columns `id`, `app_name`, and `url` are created using `Column` annotation.

```
@Entity
@Table(name = "T_APPLICATION")
public class Application {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @NotNull
    @Column(name = "ID")
    private int id;

    @NotNull
    @Column(name = "APP_NAME")
    private String appName;

    @NotNull
    @Column(name = "URL")
    private String url;
}
```

Listing 4: Java Persistence API (JPA) annotations are used define the model

e) *DTO*: A DTO (Data Transfer Object) is a Java object used to transfer data between service layers and control layers. It contains the contains fields with getters and setters, without any business logic. DTOs are used to avoid expose entities directly to outside. They improve security, reduce over fetching, and make APIs cleaner. For example, in listing 1 the controller returns a `ApplicationDTO` instead of a `Application` and in listing 2 service creates an `Application` object using the `ApplicationDTO` and returns the `ApplicationDTO` after saving. DTOs are made just like the entities but without the annotations.

f) *Mapper*: A mapper is used to convert between DTOs and entities. They keep the conversion logic out of the controller or service layers. They can be written manually as in this project or generated automatically using tools like `MapStruct` or `ModelMapper`. For example, in listing 2 service layer calls the mapper to convert between DTOs and entities instead of implementing the conversion.

All of the Controller, Service, Repository, Entity, DTO and Mapper classes are implemented for the backend.

IV. FRONTEND

The frontend of the web application built with React. React is a JavaScript library to build user interfaces. It was chosen because it has a component-based architecture, rendering to virtual DOM, and a rich ecosystem of libraries and tools. It allows quick development and maintenance. The interface is made up of reusable components. These components are used to provide functionality and presentation. This component-based structure increases the readability of the code and provides separation of concerns. The state of local variables are managed using the `useState` and `useEffect` hooks. The routing between pages was managed with React Router. CSS together with GUI component library Bootstrap used to have a clean and user-friendly design. Requests to the backend sent using the native Fetch API. The GUI was updated using the responses. Error and loading states are handled to improve user experience.

Web application has two pages, main page and add breakage page. The main page is used to show the applications, test cases, and breakage information. It has a graph and an application list part.

a) *Statistics*: In Figure 4 there is the graph on the main page showing the distal and proximal causes of the breakages, and the causes of the repairs. The options for the graph can be chosen from the buttons on the upper right. On the x-axis there are causes and on the y-axis there are the counts of the causes. At the bottom there is the explanation part. It gives information about the chosen option. For the graph `react-chartjs-2` and `Chart.js` are used.

Application Management

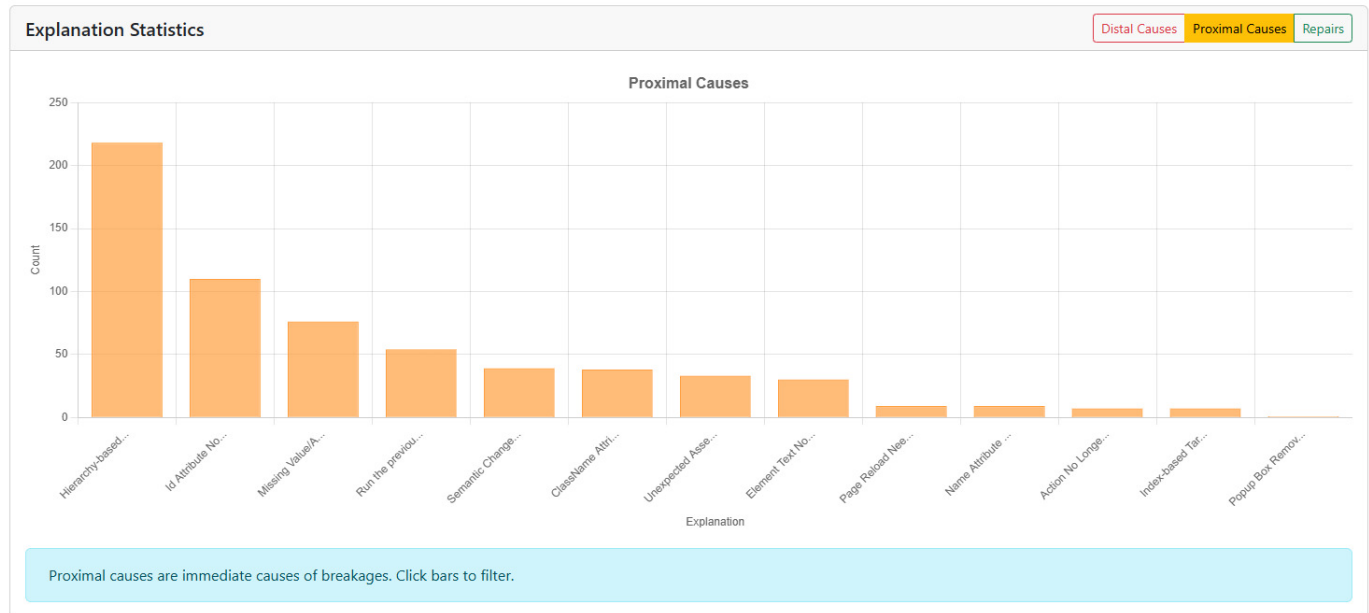
[Add Breakage](#)

Fig. 4: Main page has a graph to show the distribution of the breakages and the repairs

The figure shows a search interface titled "Search by Explanations (All Applications)". It includes a search input field with the placeholder text "Type to search explanations...". Below the input field, there is a section labeled "Selected explanations:" which contains two tags: "Element Location Changed (breakage)" with a red background and a close button, and "Fixed the Locator (repair)" with a green background and a close button. At the bottom, there is a blue button labeled "Search Across All Applications".

Fig. 5: Search breakages by their breakage causes and repairs

b) Search: In Figure 5 there is the search field on the main page. Using the search functionality, it is possible to search for breakages by their breakage causes and repair explanations. After typing the keywords, possible breakage causes and repair explanations tags are listed below the field. Simply selecting them and clicking the search button will filter the breakages in all applications according to the search criteria. Clicking on the cross on the tags will remove the search criteria. It is possible to add more tags as search criteria. The results are combined and breakages that have at least one of the criteria are listed after the search.

c) Application List: In Figure 6 there is the application list on the main page. All applications used for this study are listed. After expanding the application information about the app versions, test suite, text cases, and the breakages are listed. The breakage count shows how many breakages the app version has. For example, in Figure 6 Bludit 3.13.1 have 63 breakages and two of them can be seen listed under the test case `AddContentTest` that was originally written for Bludit 2.3.4. Under breakages, there are tags for showing the breakage causes in red and repair explanations in green.

d) Add Breakage: In Figure 7 there is the breakage form on the add breakage page. To add a breakage, the corresponding application, app version, test suite, test case, and test case version should be selected. In this example, the test case was originally written for Bludit 2.3.4. After running this test on Bludit 3.13.1 it broke. Thus, for the app version 3.13.1 and for the test case version `AddContentTest 2.3.4` is selected.

To add a new breakage, breakage reason, locating method, description, line, and the breakage explanation (cause) should be selected. After repairing the breakage, the test commit hash that contains the repair commit and the repair explanations should

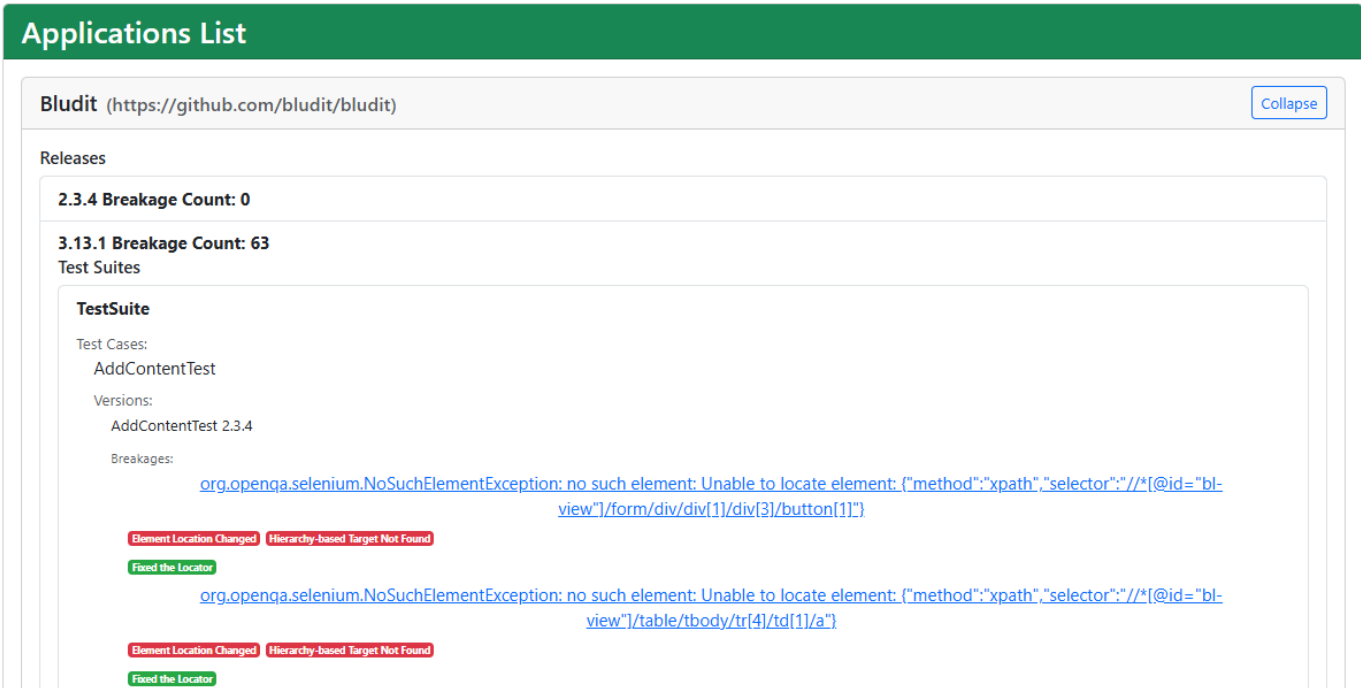


Fig. 6: Under applications test cases and breakage information

be added. By clicking the 'Add Breakage' button, breakage information is sent to the database.

There are search fields to add breakage and repair explanations. After typing the keywords, possible breakage and repair explanation tags are listed below the field. If the intended explanations are not listed, it is possible to add a new explanation by using the add new explanation field that appears below the search field. After selecting the type of explanation (proximal or distal), clicking the add button will add the new explanation. The type is not relevant for the repair explanations. It is possible to add more than one breakage and repair explanation. Simply clicking on the cross on the tags will remove it.

V. EXTENDED DESCRIPTION OF THE TAXONOMY OF TEST CASE BREAKAGE CAUSES

After applying the above described process, we obtained the taxonomy of test case breakage causes shown in ?? . It consists, at the first level, of six main categories that then further branch out.

A. Locators

Locators identify and interact with web elements such as buttons, fields, etc. In this category, breakages occur when the page structure or attributes change. It has two subcategories, namely Structure-based Locators and Attribute-based Locators.

1) *Structure-Based Locators*: Structure-based locators can rely on either the layout of elements in the DOM tree (hierarchy-based) or the position of elements using indices (index-based).

Hierarchy-Based Target Not Found. This breakage occurs when a test tries to locate an element using its location information. The test can identify and interact with an element in the old version V, but it is no longer possible in the new version V'. The distal causes of this breakage are the addition, deletion, or modification of an ancestor of the element used by the test, for example, in listing ??, if another division is added before web element `advancedTab` in the V', then the XPath of the element changes and we cannot reach it using the old XPath.

Index-Based Target Not Found. This breakage occurs when a test tries to locate an element using its location information. The test can identify and interact with an element in the old version V, but it is no longer possible in the new version V'. Distal causes of this breakage are addition, deletion, or modification of an element that is identical to the element under test in other locating strategies, e.g., items in a dropdown list, for example, in listing ??, if another button is added before web element `saveBtn` in the V', then the index of the element under test changes and we cannot reach it using the old XPath.

2) *Attribute-based Locators*: Attribute-based locators use attributes to locate specific elements. It is typically done by matching the attribute's name-value pair or targeting the element's visible text content. This category has two subcategories that concern the text of the element and the attributes of the element.

Element Text Not Found. This breakage occurs when the test tries to locate an element using its text value. In the old version, the test can reach the element using its text, but in the newer version, it is not possible since the text has changed. This causes

Breakage for AddContentTest 2.3.4

Select breakages ▾

+ Add Breakage with Repair + Add Breakage Reason + Add Locating Method

Add New Breakage

Breakage Reason

org.openqa.selenium.NoSuchElementException: no such element: Unable to locate element ▾

Locating Method

xpath ▾

Description

org.openqa.selenium.NoSuchElementException: no such element: Unable to locate element: ("method":"css selector","selector":".login-button")

Error Line

at po.NewContentPage.advanced(NewContentPage.java:54) at tests.ChangePositionTest.changePosition(ChangePositionTest.java:20)

Breakage Explanations

Search or add new breakage explanations

Element Location Changed [Detail](#) × [Hierarchy-based Target Not Found](#) [Detail](#) ×

Commit Hash

85b673eb8f1cb03265492b5bc35dd2a18a0d438b

Repair Explanations

Search or add new repair explanations

Fixed the Locator [Detail](#) ×

Add Breakage

Fig. 7: Breakage information inserted

the test to break. For example, in listing ?? you can see the driver identifies the log out button using its text value, “Logout”. If this text changes to “Log out” in the newer version of the web app, the test will break on line 22 since it will be no longer possible to locate the element using the text “Logout”.

Element Attribute Not Found. This breakage occurs when the test tries to locate an element using its attributes. In the old version, test can reach the element using one of its attributes such as id, name, and class, but in the newer version it is not possible since the attribute values are changed.

This causes the test to break. For example, in listing ?? the web element is located using its id value `name`. If this value changes to `state_name` in the newer version of the web app, the test will break where this web element object is used, since it will no longer be possible to locate the element using the id `name`. Element Attribute Not Found is subdivided in the following three categories.

3) *Id Attribute Not Found:* As explained above, this breakage occurs when the id attribute of the element changes. This change will cause the test to break since it will no longer be possible to locate the same element using the same id in the newer version.

4) *Name Attribute Not Found:* This breakage occurs when the name attribute of the element changes. This change will cause the test to break since it will no longer be possible to locate the same element using the same name in the newer version.

For example, in listing 5 the web element `deleteBtn` is located using its name value `delete`. If this value changes to `delete_btn` in the newer version of the web app, the test will break where this web element object is used, since it will no longer be possible to locate the element using the name `delete`.

```
@FindBy(name="delete")
protected WebElement deleteBtn;

@FindBy(className="uk-alert")
```

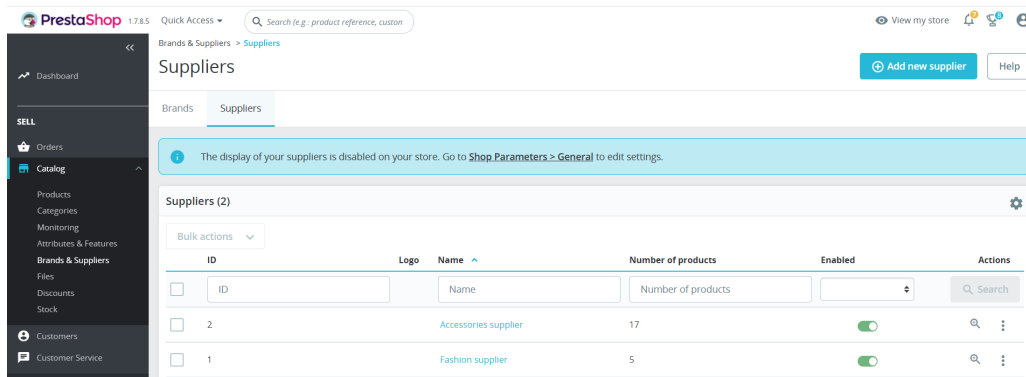


Fig. 8: PrestaShop new version Brands & Suppliers page

```
protected WebElement alert;
```

Listing 5: Code Snippet That Uses name and className

5) *ClassName Attribute Not Found*: This breakage occurs when the className attribute of the element changes. This change will cause the test to break since it will no longer be possible to locate the same element using the same className in the newer version.

For example, in listing 5 the web element `alert` is located using its className value `uk-alert`. If this value changes to `alert` in the newer version of the web app, the test will break where this web element object is used, since it will no longer be possible to locate the element using the className `uk-alert`.

B. Values/Actions

All breakages in this category are proximally related to values or actions used within the test cases themselves. Tests fail because the inputs, outputs, or expected behaviors do not match anymore. It has two subcategories Missing Value/Action and Unexpected Assertion Value.

1) *Missing Value/Action*: This breakage is caused by the actions or values in the test case that are not complete or are no longer accepted. For example, in listing 6, in the suppliers page the button `addSupplier` is clicked to go to the add supplier form. However, as can be seen in Figure 8, in the new version the suppliers page moved inside Brands & Suppliers page. To go to the suppliers page, the suppliers tab must be clicked. Otherwise, it is not possible to click on the Add new supplier button. This is why the test will break in the `clickAddSupplier` function because without clicking the suppliers tab there is no `addSupplier` is present.

```
public AddSupplierPage clickAddSupplier() {
    addSupplier.click();
    return new AddSupplierPage(driver);
}
```

Listing 6: Test clicks on the addSupplier button

2) *Action No Longer Needed*: This breakage is caused by the actions in the test case are no longer needed.

For example, in Figure 9, in the add currency page of PrestaShop web app there exists two fields required for the form to be sent, a numeric ISO code and symbol. The test case is written so that these fields are located and filled with the corresponding keys. However, as can be seen in Figure 10, the fields Numeric ISO Code and Symbol no longer exist. Thus, the test case will break when it tries to reach these fields.

3) *Unexpected Assertion Value*: This breakage is caused by the assertion values that are no longer accurate. Assertions are used to compare expected values to actual values, and to signal cases in which these do not match. When the values corresponding to an assertion value are changed, the expected value in the assertion should also change. Otherwise, breakages occur.

For example, in listing 7, line 17 checks if the alert message and the expected value 'Empty password not allowed.' equal or not. In the new version of the web app, if the alert message is changed to 'Please fill in this field', the assertion will fail because the expected value and the actual value are no longer the same. Thus, the test will fail even though there is no bug in the web app and the expected behavior is correct.

```
public class EmptySiteAdminLogin extends BaseTest {
```

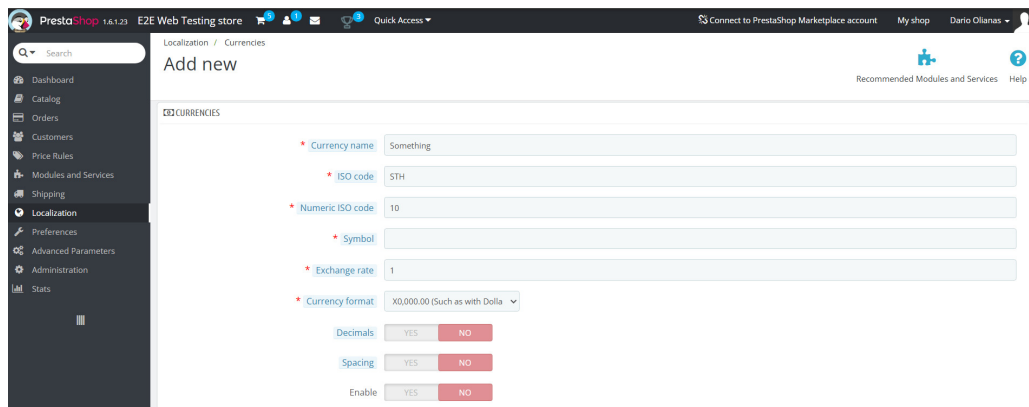


Fig. 9: Add currency page with Numeric ISO Code and Symbol in old version of PrestaShop web app

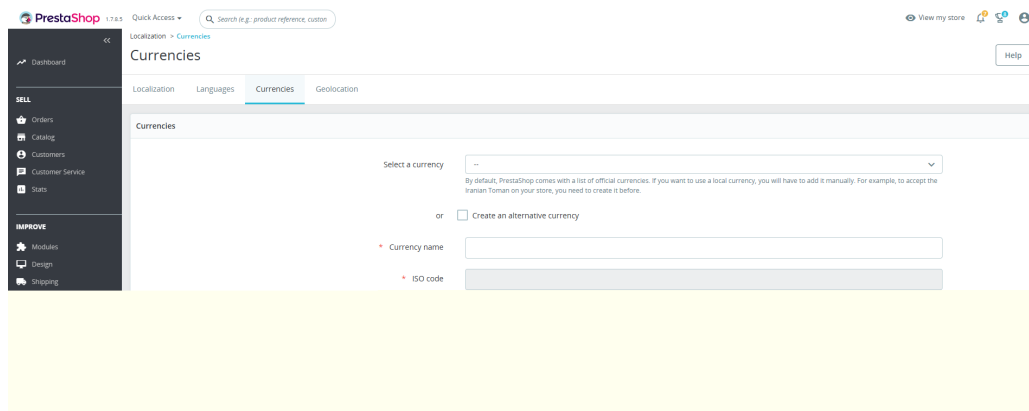


Fig. 10: Add currency page in new version of PrestaShop web app

```
@Test
public void emptySiteAdminLogin() {
    SiteAdminLogin login = loginAsAdmin()
        .siteAdmin()
        .badLogin();

    assertEquals("Empty password not allowed.", login.getAlertMessage());
}
```

Listing 7: assertEquals check if the alert message and the expected value 'Empty password not allowed.' equal or not.

C. Page Reload Needed

Breakages in this category are related to page load or timing issues. It occurs when a test case tries to interact with an element before it is present in the DOM. While the element was present without a load in the old version of the web app, in the new version the element is present after a load.

For example, in the newer version of the Bludit web app, the test should wait for homepage options to load before interacting with it. In the older version, this was not needed. In listing 8 test interacts with the element with id `jshomepage`. Before it reaches the element, it waits 1000 milliseconds for it to be loaded. If we remove the wait, the test fails.

```
public String getSelectedHome() {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    return new Select(driver.findElement(By.id("jshomepage"))).getFirstSelectedOption().getText();
}
```

```
}
```

Listing 8: Selenium WebDriver waits 1000 milliseconds before interacting the element with id jshomepage

D. Popup Box Removed

This breakage occurs when the popup box in the older version of the web app is removed. The test tries to manage the popup box in the newer version but it is not present. This causes the test to break.

For example, in listing 9 delete function interacts with the alerts after clicking the delete button. In the newer version, these alerts are removed, and thus the test breaks.

```
public UsersPage delete() {
    delete.click();
    driver.switchTo().alert().accept();
    driver.switchTo().defaultContent();
    return new UsersPage(driver);
}
```

Listing 9: Delete method interacts with the alerts after clicking delete button

E. Semantic Change In Interaction

This breakage occurs when the interaction with an element is no longer valid. For example, the Bludit web app was using a traditional dropdown to select the parent of a content. In the newer version, this traditional dropdown is changed with Select2. Select2 replaces the traditional dropdown with a custom GUI. As a result, Selenium's Select cannot interact with Select2, because it hides the original `<select>` and replaces it with custom HTML. Thus, the test breaks.

In listing 10 there is the test with the changed interaction. It does the following instead of sending the keywords directly. It clicks on the Select2 widget to activate the dropdown. Sends the key to the search input field. Waits for the dropdown to load and selects the first matching result. Another example could be a test from the PrestaShop app. In listing 11 test hovers to catalog, then hovers to product and then clicks on product button. In the newer version, the test fails in these lines because the product element was not yet visible/rendered when `moveToElement(product)` was called. Thus, hovering to it was not successful.

```
public NewContentPage setParent(String parent) {
    driver.findElement(By.xpath("//*[@id=\"nav-advanced\"] /div[5]/span/span[1]/span")).click();
    driver.findElement(By.xpath("/html/body/span/span/span[1]/input")).sendKeys(parent);
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    driver.findElement(By.xpath("//*[@id=\"select2-jsparent-results\"] /li")).click();
    return this;
}
```

Listing 10: Driver clicks on the item before sending the keys

```
public ProductsPage goToProducts() {
    Actions action = new Actions(driver);
    action.moveToElement(catalog).moveToElement(product).click().build().perform();
    return new ProductsPage(driver);
}
```

Listing 11: Driver hovers to the items

F. Dependency on Different Tests

Each test runs individually and without the changes from the previous tests. This category of breakages occurred when a test was broken because the test assumed that the changes from another were applied to the web application. The implementation of the test cases does not directly cause them, but rather their design. For example, when the `addUser` test failed to add a new user with the name John, the test `editUser`, which tries to edit the user with the name John, also fails. Counting these occurrences reveals design weaknesses that prevent E2E test cases to be executed in CI/CD pipelines.